Université Paris-Saclay / UVSQ — Master DataScale

Construction d'un Pipeline de Données Sismiques

Kafka, Logstash, ElasticStack & Spark

Données USGS Earthquake Hazards Program

Souhil OUCHENE

UE Indexation et visualisation de données massives

Février 2026

Dépôt GitHub :
https://github.com/souch19/kafka-spark-elastic-seismic-pipeline

Résumé

Ce rapport présente la conception et l'implémentation d'un pipeline de données complet pour la collecte, la transmission, la transformation, l'indexation et l'analyse de données sismiques mondiales. Le pipeline s'appuie sur l'API publique USGS Earthquake, Apache Kafka pour le streaming, Logstash et Elasticsearch pour l'ETL et l'indexation, Kibana pour la visualisation, et Apache Spark pour le traitement distribué. L'ensemble de l'infrastructure est orchestré via Docker Compose. Les données couvrent 12 mois d'activité sismique mondiale (magnitude ≥ 2.0), représentant plusieurs dizaines de milliers d'événements.

Table des matières

1 Introduction **4**
1.1 Architecture globale . . . 4
1.2 Infrastructure Docker . . . 5
1.3 Structure du projet . . . 5

2 Partie 1 : Collecte des données via l'API USGS **5**
2.1 Choix de l'API . . . 5
2.2 Script de collecte (`kafka_producer.py`) . . . 6
2.3 Normalisation des données . . . 6
2.4 Exemple de donnée extraite . . . 7

3 Partie 2 : Transmission des données avec Kafka **7**
3.1 Configuration de l'infrastructure Kafka . . . 7
3.2 Producteur Kafka . . . 8
3.3 Consommateur Kafka (debug) . . . 9
3.4 Captures Kafka . . . 9

4 Partie 3 : Transformation, indexation et requêtes **10**
4.1 Pipeline Logstash (`pipeline.conf`) . . . 10
4.2 Mapping Elasticsearch . . . 11
4.2.1 Analyzers et filtres . . . 11
4.2.2 Champs multi-field . . . 12
4.2.3 Création de l'index (`setup_elasticsearch.py`) . . . 12
4.3 Les 5 requêtes Elasticsearch . . . 14
4.3.1 Requête 1 — Recherche textuelle avancée . . . 14
4.3.2 Requête 2 — Agrégation pipeline . . . 16
4.3.3 Requête 3 — Recherche N-gram + agrégations . . . 17
4.3.4 Requête 4 — Recherche fuzzy avec `function_score` . . . 18
4.3.5 Requête 5 — Série temporelle avancée . . . 19

5 Partie 4 : Visualisation avec Kibana **20**
5.1 Carte mondiale des séismes . . . 20
5.2 Histogramme temporel par type de magnitude . . . 21
5.3 Top régions sismiques — Alaska . . . 22
5.4 Couverture des réseaux sismiques par magnitude . . . 22
5.5 Alertes tsunami et magnitude maximale (hebdomadaire) . . . 23

6 Partie 5 : Traitement distribué avec Spark **25**
6.1 Justification du choix de Spark . . . 25
6.2 Architecture technique . . . 25
6.3 Les 5 analyses Spark . . . 25
6.3.1 Analyse 1 — Statistiques globales . . . 26
6.3.2 Analyse 2 — Analyse par région . . . 26
6.3.3 Analyse 3 — Analyse temporelle avec moyenne mobile . . . 27
6.3.4 Analyse 4 — Corrélation magnitude / profondeur . . . 28
6.3.5 Analyse 5 — Analyse par type de magnitude . . . 29

6.4 Export des résultats 30
6.5 Commande d'exécution Spark 30

7 Documentation, organisation et choix techniques 31
7.1 Gestion de la robustesse 31
7.2 Choix techniques justifiés 31
7.3 Dépôt de code 31

8 Conclusion 32

A Annexe A — Commandes de gestion du pipeline 32

B Annexe B — Récapitulatif des fichiers livrés 33

1 Introduction

L'objectif de ce projet est de construire un pipeline de données end-to-end capable de gérer un flux continu de données structurées. Nous avons choisi de travailler avec les données sismiques du programme USGS Earthquake Hazards, qui offrent plusieurs avantages : des données temporelles riches, des champs textuels exploitables pour les recherches full-text et floues, des coordonnées géographiques pour la cartographie, et des métriques numériques (magnitude, profondeur) adaptées aux agrégations et aux traitements distribués.

1.1 Architecture globale

L'architecture suit un modèle de streaming pipeline en cinq étapes. La figure 1 illustre le flux de données.

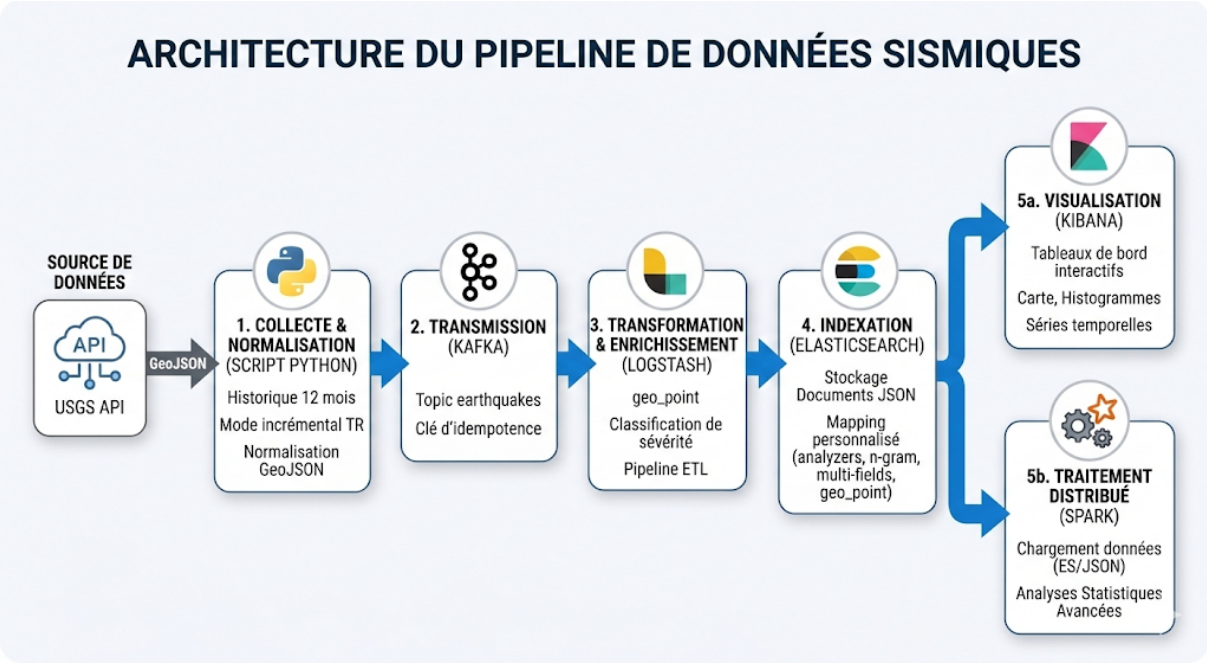

FIGURE 1 – Architecture du pipeline de données sismiques

1. **Collecte** : un script Python interroge l'API USGS sur 12 mois d'historique, normalise les données GeoJSON en schéma plat, puis bascule en mode incrémental temps réel.
2. **Transmission** : les événements sont publiés dans un topic Kafka (`earthquakes`) avec clé d'idempotence.
3. **Transformation** : Logstash consomme le topic, enrichit les données (geo_point, classification de sévérité) et les indexe dans Elasticsearch.
4. **Indexation** : Elasticsearch stocke les documents avec un mapping personnalisé (analyzers custom, n-gram, multi-fields, geo_point).
5. **Visualisation** : Kibana exploite l'index pour créer des tableaux de bord interactifs (carte, histogrammes, séries temporelles).
6. **Traitement distribué** : Spark charge les données depuis ES (ou fallback JSON) et effectue 5 analyses statistiques avancées.

1.2 Infrastructure Docker

L'ensemble des services est conteneurisé dans un fichier `docker-compose.yml`. Un réseau Docker dédié (`pipeline-network`) assure la communication inter-services. Le tableau 1 résume les composants.

Table 1 – Services Docker déployés

Service	Image	Port	Rôle
Zookeeper	confluentinc/cp-zookeeper :7.5.0	2181	Coordination Kafka
Kafka	confluentinc/cp-kafka :7.5.0	9092, 29092	Broker de messages
Elasticsearch	elastic/elasticsearch :8.11.0	9200	Indexation & recherche
Logstash	elastic/logstash :8.11.0	5044	Pipeline ETL
Kibana	elastic/kibana :8.11.0	5601	Visualisation
Spark Master	apache/spark :3.5.1	8080, 7077	Traitement distribué
Spark Worker	apache/spark :3.5.1	—	Exécution des tâches

1.3 Structure du projet

```
pipeline-earthquakes/
  docker-compose.yml
  README.md
  config/
    elasticsearch/mapping.json
    logstash/pipeline.conf
  scripts/
    kafka_producer.py          # Collecte API + Production Kafka
    kafka_consumer.py          # Consommateur debug
    setup_elasticsearch.py     # Creation index + mapping
    export_es_to_json.py       # Export ES -> JSON (fallback Spark)
    requirements.txt
  queries/
    run_queries.py             # 5 requetes ES + sauvegarde resultats
  spark/
    spark_processing.py        # 5 analyses distribuees PySpark
  results/
    queries/                   # Resultats des 5 requetes ES
    spark/                     # Resultats Spark (JSON + CSV)
  screenshots/                 # Captures d'ecran
  rapport/
    rapport.pdf
```

Listing 1 – Arborescence du projet

2 Partie 1 : Collecte des données via l'API USGS

2.1 Choix de l'API

L'API **USGS Earthquake Hazards Program**[1] est une API REST publique sans authentification. Elle retourne des données au format GeoJSON contenant pour chaque séisme : magnitude, coordonnées géographiques, profondeur, horodatage, lieu, type de magnitude, alerte tsunami, et métadonnées supplémentaires (réseau sismique, RMS, gap azimutal, etc.).

1. https://earthquake.usgs.gov/fdsnws/event/1/

Avantages pour le projet :

- Données temporelles riches (champ `time`) pour les séries temporelles et histogrammes.
- Champs textuels (`place`, `title`) adaptés aux recherches full-text, n-gram et fuzzy.
- Coordonnées géographiques pour les cartes Kibana (`geo_point`).
- Métriques numériques (magnitude, profondeur, sig) pour les agrégations et traitements Spark.
- Volume important : ∼600 événements/jour avec magnitude ≥ 2.0.

2.2 Script de collecte (`kafka_producer.py`)

Le script combine collecte API et production Kafka en deux phases :

Phase 1 — Chargement historique : 12 mois d'historique, paginé par tranches mensuelles pour respecter la limite de 20 000 événements par requête API. Aucune limite artificielle n'est imposée via le paramètre `limit`.

Phase 2 — Mode incrémental : boucle temps réel interrogeant l'API toutes les 60 secondes pour les nouveaux événements.

```python
HISTORY_MONTHS = 12
MIN_MAGNITUDE_HISTORY = 2.0
MIN_MAGNITUDE_LIVE = 1.0

def fetch_earthquakes(start_time, end_time, min_magnitude=1.0):
    """Aucune limite -- l'API renvoie jusqu'a 20 000 evenements."""
    params = {
        "format": "geojson",
        "starttime": start_time,
        "endtime": end_time,
        "minmagnitude": min_magnitude,
        "orderby": "time",
    }
    resp = requests.get(USGS_API_URL, params=params, timeout=60)
    resp.raise_for_status()
    return [normalize_event(f) for f in resp.json().get("features", [])]

def generate_monthly_ranges(months_back):
    now = datetime.utcnow()
    ranges = []
    for i in range(months_back, 0, -1):
        start = now - timedelta(days=i * 30)
        end = now - timedelta(days=(i - 1) * 30)
        ranges.append((start.strftime("%Y-%m-%d"), end.strftime("%Y-%m-%d")))
    return ranges

def bulk_load_history(producer):
    ranges = generate_monthly_ranges(HISTORY_MONTHS)
    total = 0
    for start, end in ranges:
        events = fetch_earthquakes(start, end, MIN_MAGNITUDE_HISTORY)
        for event in events:
            producer.send(KAFKA_TOPIC, key=event["id"], value=event)
        producer.flush()
        total += len(events)
        time.sleep(0.5)
    return total
```

Listing 2 – Collecte paginée par tranches mensuelles

2.3 Normalisation des données

Les données brutes GeoJSON sont normalisées en un schéma plat, adapté à l'indexation ES et au traitement Spark. Les coordonnées géographiques imbriquées dans

`geometry.coordinates` sont extraites en champs séparés `longitude`, `latitude`, `depth`. Les horodatages epoch (millisecondes) sont convertis en ISO 8601.

```python
def normalize_event(feature):
    props = feature["properties"]
    coords = feature["geometry"]["coordinates"]
    return {
        "id": feature["id"],
        "magnitude": props.get("mag"),
        "place": props.get("place", ""),
        "time": datetime.utcfromtimestamp(props["time"] / 1000).isoformat() + "Z",
        "updated": datetime.utcfromtimestamp(props["updated"] / 1000).isoformat() + "Z",
        "type": props.get("type", "earthquake"),
        "title": props.get("title", ""),
        "status": props.get("status", ""),
        "tsunami": props.get("tsunami", 0),
        "sig": props.get("sig", 0),
        "net": props.get("net", ""),
        "nst": props.get("nst"),
        "dmin": props.get("dmin"),
        "rms": props.get("rms"),
        "gap": props.get("gap"),
        "mag_type": props.get("magType", ""),
        "longitude": coords[0],
        "latitude": coords[1],
        "depth": coords[2],
        "url": props.get("url", ""),
    }
```

Listing 3 – Normalisation d'un événement GeoJSON en schéma plat

2.4 Exemple de donnée extraite

```json
{
  "id": "us7000n1ab",
  "magnitude": 5.2,
  "place": "45 km NE of Hualien City, Taiwan",
  "time": "2025-02-10T14:23:45Z",
  "updated": "2025-02-10T15:10:12Z",
  "type": "earthquake",
  "title": "M 5.2 - 45 km NE of Hualien City, Taiwan",
  "status": "reviewed",
  "tsunami": 0,
  "sig": 416,
  "net": "us",
  "nst": 120,
  "dmin": 1.234,
  "rms": 0.89,
  "gap": 32.0,
  "mag_type": "mww",
  "longitude": 121.8234,
  "latitude": 24.2156,
  "depth": 12.5,
  "url": "https://earthquake.usgs.gov/earthquakes/eventpage/us7000n1ab"
}
```

Listing 4 – Exemple d'événement sismique normalisé (JSON)

3 Partie 2 : Transmission des données avec Kafka

3.1 Configuration de l'infrastructure Kafka

Kafka est déployé via Docker Compose avec Zookeeper comme coordinateur. Deux listeners sont configurés pour gérer la communication interne (conteneurs) et externe

(hôte) :

— `PLAINTEXT://kafka:9092` : communication inter-conteneurs (Logstash accède à Kafka via ce listener).
— `PLAINTEXT_HOST://localhost:29092` : accès depuis l'hôte pour les scripts Python.

Le topic `earthquakes` est créé automatiquement lors du premier envoi grâce à `KAFKA_AUTO_CREATE_TOPICS_ENABLE=true`.

```yaml
zookeeper:
  image: confluentinc/cp-zookeeper:7.5.0
  environment:
    ZOOKEEPER_CLIENT_PORT: 2181
    ZOOKEEPER_TICK_TIME: 2000
  ports:
    - "2181:2181"

kafka:
  image: confluentinc/cp-kafka:7.5.0
  depends_on:
    - zookeeper
  ports:
    - "9092:9092"
    - "29092:29092"
  environment:
    KAFKA_BROKER_ID: 1
    KAFKA_ZOOKEEPER_CONNECT: zookeeper:2181
    KAFKA_ADVERTISED_LISTENERS: >
      PLAINTEXT://kafka:9092,
      PLAINTEXT_HOST://localhost:29092
    KAFKA_LISTENER_SECURITY_PROTOCOL_MAP: >
      PLAINTEXT:PLAINTEXT,PLAINTEXT_HOST:PLAINTEXT
    KAFKA_INTER_BROKER_LISTENER_NAME: PLAINTEXT
    KAFKA_OFFSETS_TOPIC_REPLICATION_FACTOR: 1
    KAFKA_AUTO_CREATE_TOPICS_ENABLE: "true"
```

Listing 5 – Configuration Docker Compose — Kafka & Zookeeper

3.2 Producteur Kafka

Le producteur utilise `kafka-python` avec sérialisation JSON. Chaque événement est envoyé avec une clé correspondant à l'identifiant unique du séisme (`event["id"]`), ce qui garantit l'idempotence lors des réexécutions. Un mécanisme de retry exponentiel gère le démarrage asynchrone du broker.

```python
def create_producer(bootstrap_servers, retries=10):
    for attempt in range(retries):
        try:
            producer = KafkaProducer(
                bootstrap_servers=bootstrap_servers,
                value_serializer=lambda v: json.dumps(v).encode("utf-8"),
                key_serializer=lambda k: k.encode("utf-8") if k else None,
                acks="all",
                retries=3,
            )
            print(f"[OK] Connecte a Kafka ({bootstrap_servers})")
            return producer
        except NoBrokersAvailable:
            wait = min(2 ** attempt, 30)
            print(f"[WAIT] Kafka indisponible, tentative dans {wait}s...")
            time.sleep(wait)
    print("[ERREUR] Impossible de se connecter")
    sys.exit(1)
```

Listing 6 – Création du producteur Kafka avec retry exponentiel

3.3 Consommateur Kafka (debug)

Un consommateur standalone (`kafka_consumer.py`) est fourni pour le débogage et la validation des messages transmis. Il lit depuis l'offset le plus ancien

(`auto_offset_reset="earliest"`) et sauvegarde un échantillon tous les 100 messages. En production, c'est Logstash qui consomme le topic.

```
consumer = KafkaConsumer(
    "earthquakes",
    bootstrap_servers="localhost:29092",
    group_id="earthquake-debug-consumer",
    auto_offset_reset="earliest",
    value_deserializer=lambda m: json.loads(m.decode("utf-8")),
    consumer_timeout_ms=10000,
)
```

Listing 7 – Consommateur Kafka — configuration

3.4 Captures Kafka

```
PS C:\Users\souhi\Desktop\projet-pipeline-earthquakes\Projet_indexation_OUCH
ENE_SOUHIL\kafka-spark-elastic-seismic-pipeline> docker exec kafka kafka-top
ics --list --bootstrap-server localhost:9092
__consumer_offsets
earthquakes
PS C:\Users\souhi\Desktop\projet-pipeline-earthquakes\Projet_indexation_OUCH
ENE_SOUHIL\kafka-spark-elastic-seismic-pipeline> docker exec kafka kafka-top
ics --describe --topic earthquakes --bootstrap-server localhost:9092
Topic: earthquakes      TopicId: 0NvXXnt4SxeYQQ0YWBgayg PartitionCount: 1  R
eplicationFactor: 1     Configs:
        Topic: earthquakes      Partition: 0    Leader: 1       Replicas: 1I
sr: 1
PS C:\Users\souhi\Desktop\projet-pipeline-earthquakes\Projet_indexation_OUCH
ENE_SOUHIL\kafka-spark-elastic-seismic-pipeline> docker exec kafka kafka-con
sumer-groups --list --bootstrap-server localhost:9092
logstash-earthquake-consumer
```

FIGURE 2 – Captures Kafka : producteur, topic et consommateur debug

```
PS C:\Users\souhi\Desktop\projet-pipeline-earthquakes\Projet_indexation_OUCH
ENE_SOUHIL\kafka-spark-elastic-seismic-pipeline> python scripts/kafka_produc
er.py
============================================================
  USGS Earthquake Collector -> Kafka Producer
  Historique : 12 mois | Magnitude >= 2.0
============================================================
[OK] Connecte a Kafka (localhost:29092)
[INIT] Chargement historique : 12 mois (12 tranches)
  [2025-03-06 -> 2025-04-05] 3246 evenements (total: 3246)
  [2025-04-05 -> 2025-05-05] 3108 evenements (total: 6354)
  [2025-05-05 -> 2025-06-04] 2863 evenements (total: 9217)
  [2025-06-04 -> 2025-07-04] 3023 evenements (total: 12240)
  [2025-07-04 -> 2025-08-03] 6327 evenements (total: 18567)
  [2025-08-03 -> 2025-09-02] 3872 evenements (total: 22439)
  [2025-09-02 -> 2025-10-02] 2879 evenements (total: 25318)
  [2025-10-02 -> 2025-11-01] 2675 evenements (total: 27993)
  [2025-11-01 -> 2025-12-01] 2929 evenements (total: 30922)
  [2025-12-01 -> 2025-12-31] 5556 evenements (total: 36478)
  [2025-12-31 -> 2026-01-30] 3198 evenements (total: 39676)
  [2026-01-30 -> 2026-03-01] 2978 evenements (total: 42654)
[INIT] 42654 evenements historiques envoyes dans 'earthquakes'

[LIVE] Passage en mode incremental (intervalle: 60s)
```

FIGURE 3 – Exécution du producteur Kafka — chargement historique de 42 654 événements sur 12 mois puis passage en mode incrémental.

4 Partie 3 : Transformation, indexation et requêtes

4.1 Pipeline Logstash (`pipeline.conf`)

Logstash est configuré pour consommer le topic Kafka, appliquer trois transformations clés, puis indexer dans Elasticsearch avec l'identifiant unique du séisme comme `document_id` (garantie d'idempotence : pas de doublons en cas de re-indexation).

Transformations appliquées :

1. **Parsing temporel :** conversion du champ ISO 8601 `time` en `@timestamp` Elasticsearch pour les séries temporelles.
2. **Création du geo_point :** construction du champ `location` (type `geo_point`) à partir de `latitude` et `longitude` pour les cartes Kibana.
3. **Classification de sévérité :** enrichissement conditionnel basé sur la magnitude (≥ 7 : major, ≥ 5 : moderate, ≥ 3 : light, sinon : minor).

```
input {
  kafka {
    bootstrap_servers => "kafka:9092"
    topics            => ["earthquakes"]
    group_id          => "logstash-earthquake-consumer"
    codec             => "json"
    auto_offset_reset => "earliest"
    consumer_threads  => 1
    decorate_events   => "basic"
  }
}

filter {
  date {
    match  => ["time", "ISO8601"]
    target => "@timestamp"
```

```
  }

  if [latitude] and [longitude] {
    mutate {
      add_field => {
        "[location][lat]" => "%{latitude}"
        "[location][lon]" => "%{longitude}"
      }
    }
    mutate {
      convert => {
        "[location][lat]" => "float"
        "[location][lon]" => "float"
      }
    }
  }

  if [magnitude] {
    if [magnitude] >= 7 {
      mutate { add_field => { "severity" => "major" } }
    } else if [magnitude] >= 5 {
      mutate { add_field => { "severity" => "moderate" } }
    } else if [magnitude] >= 3 {
      mutate { add_field => { "severity" => "light" } }
    } else {
      mutate { add_field => { "severity" => "minor" } }
    }
  }

  mutate {
    remove_field => ["@version", "[event][original]", "event"]
  }
}

output {
  elasticsearch {
    hosts       => ["http://elasticsearch:9200"]
    index       => "earthquakes"
    document_id => "%{id}"
    action      => "index"
  }
  stdout { codec => dots }
}
```

Listing 8 – Pipeline Logstash complet (`config/logstash/pipeline.conf`)

4.2 Mapping Elasticsearch

Le mapping définit des types explicites pour chaque champ et intègre deux analyzers personnalisés, permettant de supporter différents types de requêtes sur les mêmes données textuelles.

4.2.1 Analyzers et filtres

— **text_analyzer** : tokenisation standard + lowercase + filtres stop words anglais *et* français, pour les recherches full-text.
— **ngram_analyzer** : tokenisation standard + lowercase + filtre n-gram (min=3, max=4), pour la recherche partielle.

```
"analysis": {
  "filter": {
    "ngram_filter": {
      "type": "ngram", "min_gram": 3, "max_gram": 4
    },
    "english_stop": {
      "type": "stop", "stopwords": "_english_"
```

```
      },
      "french_stop": {
        "type": "stop", "stopwords": "_french_"
      }
    },
    "analyzer": {
      "ngram_analyzer": {
        "type": "custom",
        "tokenizer": "standard",
        "filter": ["lowercase", "ngram_filter"]
      },
      "text_analyzer": {
        "type": "custom",
        "tokenizer": "standard",
        "filter": ["lowercase", "english_stop", "french_stop"]
      }
    }
  }
```

Listing 9 – Définition des analyzers dans le mapping

4.2.2 Champs multi-field

Le champ `place` est indexé avec trois variantes (multi-field) permettant différents usages selon le contexte de requête :

Table 2 – Variantes multi-field du champ `place`

Sous-champ	Type	Usage
`place`	text (text_analyzer)	Recherche full-text
`place.keyword`	keyword	Agrégations exactes, tri
`place.ngram`	text (ngram_analyzer)	Recherche partielle

Le champ `location` est typé `geo_point` pour les visualisations cartographiques Kibana. Le champ `time` est typé `date` avec format dual
(`strict_date_optional_time||epoch_millis`).

```
"place": {
  "type": "text", "analyzer": "text_analyzer",
  "fields": {
    "keyword": {"type": "keyword"},
    "ngram":   {"type": "text", "analyzer": "ngram_analyzer"}
  }
},
"location":  {"type": "geo_point"},
"time":      {"type": "date",
              "format": "strict_date_optional_time||epoch_millis"},
"magnitude": {"type": "float"},
"depth":     {"type": "float"},
"tsunami":   {"type": "integer"},
"mag_type":  {"type": "keyword"},
"status":    {"type": "keyword"}
```

Listing 10 – Mapping — champs clés (extrait)

4.2.3 Création de l'index (`setup_elasticsearch.py`)

Le script attend qu'Elasticsearch soit prêt (health check avec timeout de 120s), supprime l'index existant si présent, crée l'index avec le mapping complet, et sauvegarde une copie du mapping en JSON.

```python
def wait_for_es(host, timeout=120):
    start = time.time()
    while time.time() - start < timeout:
        try:
            r = requests.get(f"{host}/_cluster/health", timeout=5)
            if r.status_code == 200:
                print(f"[OK] Elasticsearch pret ({r.json()['status']})")
                return
        except requests.ConnectionError:
            pass
        time.sleep(2)
    sys.exit(1)

def create_index():
    wait_for_es(ES_HOST)
    requests.delete(f"{ES_HOST}/{INDEX_NAME}")
    resp = requests.put(
        f"{ES_HOST}/{INDEX_NAME}",
        headers={"Content-Type": "application/json"},
        json=MAPPING
    )
```

Listing 11 – Création de l'index avec attente du cluster

```
PS C:\Users\souhi\Desktop\projet-pipeline-earthquakes\Projet_indexation_OUCH
ENE_SOUHIL\kafka-spark-elastic-seismic-pipeline> curl.exe http://localhost:9
200/_cluster/health?pretty
{
  "cluster_name" : "docker-cluster",
  "status" : "green",
  "timed_out" : false,
  "number_of_nodes" : 1,
  "number_of_data_nodes" : 1,
  "active_primary_shards" : 30,
  "active_shards" : 30,
  "relocating_shards" : 0,
  "initializing_shards" : 0,
  "unassigned_shards" : 0,
  "delayed_unassigned_shards" : 0,
  "number_of_pending_tasks" : 0,
  "number_of_in_flight_fetch" : 0,
  "task_max_waiting_in_queue_millis" : 0,
  "active_shards_percent_as_number" : 100.0
}
PS C:\Users\souhi\Desktop\projet-pipeline-earthquakes\Projet_indexation_OUCH
ENE_SOUHIL\kafka-spark-elastic-seismic-pipeline> curl.exe http://localhost:9
200/earthquakes/_count
{"count":42635,"_shards":{"total":1,"successful":1,"skipped":0,"failed":0}}
```

FIGURE 4 – Santé du cluster Elasticsearch et nombre de documents indexés

```
PS C:\Users\souhi\Desktop\projet-pipeline-earthquakes\Projet_indexation_OUCH
ENE_SOUHIL\kafka-spark-elastic-seismic-pipeline> python scripts/setup_elasti
csearch.py
Connexion a http://localhost:9200...
[OK] Elasticsearch est pret (green)
[OK] Index 'earthquakes' cree avec succes
{
  "acknowledged": true,
  "shards_acknowledged": true,
  "index": "earthquakes"
}
[OK] Mapping sauvegarde dans C:\Users\souhi\Desktop\projet-pipeline-earthqua
kes\Projet_indexation_OUCHENE_SOUHIL\kafka-spark-elastic-seismic-pipeline\co
nfig\elasticsearch\mapping.json
```

FIGURE 5 – Création de l'index Elasticsearch — cluster en statut green, index « earthquakes » créé avec succès et mapping sauvegardé.

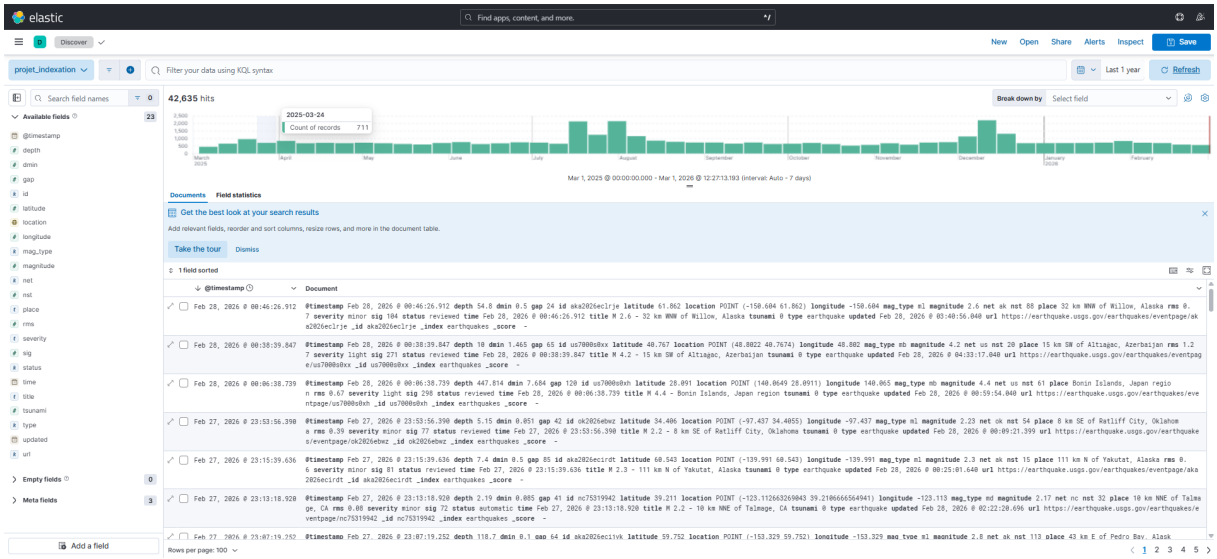

FIGURE 6 – Kibana Discover — données indexées avec champs enrichis (severity, location)

4.3 Les 5 requêtes Elasticsearch

Le script `queries/run_queries.py` exécute les 5 requêtes demandées et sauvegarde pour chacune : la requête (`*_request.json`), la réponse complète (`*_response.json`), et un résumé lisible (`*_resume.txt`). L'ensemble des résultats détaillés est disponible sur le dépôt GitHub [2].

4.3.1 Requête 1 — Recherche textuelle avancée

Requête `bool` multi-champ avec boosting : recherche « California » sur `place` (×3) et `title` (×2), filtrée sur magnitude ≥ 2.5 et type `earthquake`. Les résultats avec tsunami ou forte magnitude sont boostés. Le highlighting est activé.

```
1 {
2   "size": 15,
```

2. https://github.com/souch19/kafka-spark-elastic-seismic-pipeline/tree/main/results/queries

```
  "query": {
    "bool": {
      "must": [{
        "multi_match": {
          "query": "California",
          "fields": ["place^3", "title^2"],
          "type": "best_fields"
        }
      }],
      "filter": [
        {"range": {"magnitude": {"gte": 2.5}}},
        {"term": {"type": "earthquake"}}
      ],
      "should": [
        {"range": {"magnitude": {"gte": 4.5, "boost": 2.0}}},
        {"term": {"tsunami": {"value": 1, "boost": 3.0}}}
      ],
      "minimum_should_match": 0
    }
  },
  "highlight": {
    "fields": {
      "place": {"number_of_fragments": 1},
      "title": {"number_of_fragments": 1}
    }
  },
  "sort": [{"_score": "desc"}, {"magnitude": "desc"}],
  "_source": ["title", "magnitude", "place", "time", "depth", "tsunami"]
}
```

Listing 12 – Requête 1 — bool multi-champ + boosting + highlight

Résultat — Requête 1 : Textuelle (69 ms, 65 résultats)

```
  1. [score=19.12] M2.7 | off the coast of Northern California    | prof=10km
  2. [score=17.57] M3.8 | 163 km W of Ferndale, California       | prof=10km
  3. [score=17.57] M3.3 | 235 km W of Ferndale, California       | prof=10km
  4. [score=17.57] M3.2 | 155 km W of Ferndale, California       | prof=10km
  5. [score=17.57] M3.2 | 175 km W of Ferndale, California       | prof=10km
  6. [score=17.57] M3.2 | 249 km W of Ferndale, California       | prof=10km
  7. [score=17.57] M3.1 | 234 km W of Ferndale, California       | prof=10km
  8. [score=17.57] M3.1 | 152 km W of Ferndale, California       | prof=10km
  9. [score=17.57] M3.0 | 15 km NW of Fillmore, California       | prof=12km
 10. [score=17.57] M3.0 | 130 km W of Ferndale, California       | prof=10km
```

Résultats tronqués — 65 résultats au total. Détails complets sur le dépôt GitHub.

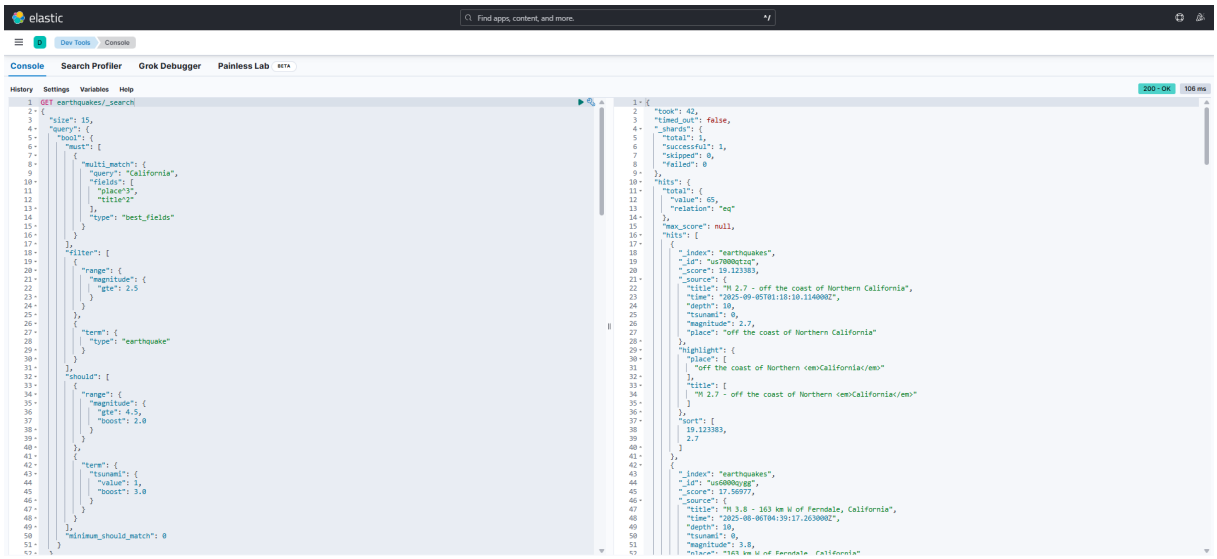

FIGURE 7 – Exécution de la requête 1 dans Kibana Dev Tools »

4.3.2 Requête 2 — Agrégation pipeline

Agrégation à trois niveaux : (1) statistiques étendues par type de magnitude avec percentiles et cardinalité des lieux, (2) agrégation par tranche de profondeur avec alertes tsunami, (3) percentiles globaux de magnitude.

```
{
  "size": 0,
  "aggs": {
    "par_mag_type": {
      "terms": {"field": "mag_type", "size": 10},
      "aggs": {
        "stats_magnitude": {"extended_stats": {"field": "magnitude"}},
        "percentiles_magnitude": {
          "percentiles": {"field": "magnitude",
                          "percents": [25, 50, 75, 90, 99]}
        },
        "profondeur_moyenne": {"avg": {"field": "depth"}},
        "nb_regions_distinctes": {"cardinality": {"field": "place.keyword"}}
      }
    },
    "par_tranche_profondeur": {
      "range": {
        "field": "depth",
        "ranges": [
          {"key": "superficiel (0-10 km)",      "from": 0,   "to": 10},
          {"key": "peu profond (10-70 km)",     "from": 10,  "to": 70},
          {"key": "intermediaire (70-300 km)", "from": 70,  "to": 300},
          {"key": "profond (300+ km)",          "from": 300}
        ]
      },
      "aggs": {
        "mag_moyenne": {"avg": {"field": "magnitude"}},
        "mag_max": {"max": {"field": "magnitude"}}
      }
    }
  }
}
```

Listing 13 – Requête 2 — agrégation multi-niveaux (extrait)

Résultat — Requête 2 : Agrégations (285 ms, 42 635 docs)

Par tranche de profondeur :

Tranche	Docs	Mag. max	Mag. moy.
Superficiel (0–10 km)	10 413	7.60	2.48
Peu profond (10–70 km)	24 906	8.80	3.57
Intermédiaire (70–300 km)	5 985	6.70	3.62
Profond (300+ km)	1 065	7.10	4.38

Par type de magnitude (top 5) :

Type	Docs	Prof. moy. (km)	Régions
ml	20 445	26.46	9 837
mb	15 778	76.83	10 266
md	4 348	20.13	2 714
mww	1 399	51.26	1 120
mwr	376	27.60	360

Résultats tronqués (10 types au total) — détails complets sur le dépôt GitHub.

4.3.3 Requête 3 — Recherche N-gram + agrégations

Recherche textuelle sur `place` pour « Alaska » avec agrégations imbriquées : statistiques étendues de magnitude, top 10 des lieux exacts (via `place.keyword`), et distribution par tranches de magnitude (histogramme à intervalle 0.5).

```
{
  "size": 50,
  "query": {
    "bool": {
      "must": [{
        "match": {
          "place": {"query": "Alaska", "operator": "or"}
        }
      }],
      "filter": [{"range": {"magnitude": {"gte": 2.0}}}]
    }
  },
  "aggs": {
    "mag_stats_alaska": {"extended_stats": {"field": "magnitude"}},
    "top_lieux": {"terms": {"field": "place.keyword", "size": 10}},
    "mag_distribution": {
      "histogram": {"field": "magnitude", "interval": 0.5}
    }
  },
  "highlight": {"fields": {"place": {"number_of_fragments": 1}}},
  "sort": [{"magnitude": "desc"}],
  "_source": ["title", "magnitude", "place", "time", "depth",
              "latitude", "longitude"]
}
```

Listing 14 – Requête 3 — N-gram Alaska + agrégations

Résultat — Requête 3 : N-gram Alaska (61 ms, 15 242 hits)

Top 5 résultats (triés par magnitude) :

```
1. M7.3 | 2025 Sand Point, Alaska Earthquake        | prof=38km
2. M6.2 | 88 km SSE of Adak, Alaska                 | prof=17km
3. M6.2 | 91 km SSE of Sand Point, Alaska            | prof=40km
4. M6.1 | 93 km SW of Nikolski, Alaska              | prof=14km
5. M6.0 | 12 km WNW of Susitna, Alaska              | prof=69km
```

Distribution par tranche de magnitude :

2.0	2.5	3.0	3.5	4.0	4.5	5.0+
8 561	3 789	1 774	670	280	104	64

Top lieu : Rat Islands, Aleutian Islands, Alaska (236 docs).

Résultats tronqués — détails complets sur le dépôt GitHub.

4.3.4 Requête 4 — Recherche fuzzy avec `function_score`

Requête multi-champ avec `fuzziness: AUTO` (distance d'édition de Levenshtein automatique) sur plusieurs termes (California, Alaska, Oklahoma, seismic earthquake). Un `function_score` multiplie le score par $\sqrt{\text{magnitude} \times 1.2}$ pour privilégier les événements significatifs.

```
{
  "size": 20,
  "query": {
    "function_score": {
      "query": {
        "bool": {
          "should": [
            {"match": {"place": {
              "query": "California", "fuzziness": "AUTO", "boost": 2.0
            }}},
            {"match": {"place": {
              "query": "Alaska", "fuzziness": "AUTO", "boost": 1.5
            }}},
            {"match": {"place": {
              "query": "Oklahoma", "fuzziness": "AUTO"
            }}},
            {"match": {"title": {
              "query": "seismic earthquake", "fuzziness": "AUTO",
              "boost": 1.2
            }}}
          ],
          "minimum_should_match": 1,
          "filter": [{"range": {"magnitude": {"gte": 1.5}}}]
        }
      },
      "functions": [{
        "field_value_factor": {
          "field": "magnitude",
          "factor": 1.2,
          "modifier": "sqrt",
          "missing": 1
        }
      }],
      "boost_mode": "multiply"
    }
  },
  "highlight": {
    "fields": {"place": {}, "title": {}}
  }
}
```

Listing 15 – Requête 4 — fuzzy + function_score (extrait)

Résultat — Requête 4 : Fuzzy + function_score (415 ms, 10 000+ hits)

Le `function_score` avec $\sqrt{\text{magnitude} \times 1.2}$ remonte les séismes majeurs en tête :

```
  1. [score=34.61] M7.3 | Sand Point, Alaska Earthquake
  2. [score=32.96] M8.8 | Kamchatka Peninsula, Russia Earthquake
  3. [score=32.42] M7.4 | Drake Passage Earthquake
  4. [score=31.53] M7.0 | Hubbard Glacier Earthquake
  5. [score=30.83] M7.7 | Mandalay, Burma (Myanmar) Earthquake
  6. [score=30.63] M7.6 | Aomori Prefecture, Japan Earthquake
  7. [score=30.43] M7.5 | Southern Drake Passage Earthquake
  8. [score=30.22] M7.4 | Eastern Kamchatka, Russia Earthquake
  9. [score=29.19] M6.0 | Taitung, Taiwan Earthquake
 10. [score=25.01] M3.8 | 163 km W of Ferndale, California
```

Résultats tronqués — détails complets sur le dépôt GitHub.

4.3.5 Requête 5 — Série temporelle avancée

Histogramme hebdomadaire avec sous-agrégations riches : magnitude moyenne/max/-min, profondeur moyenne, nombre de séismes significatifs ($M \geq 4$), répartition par type de magnitude, moyenne mobile sur 3 semaines (`moving_fn`), et somme cumulative (`cumulative_sum`). Une seconde agrégation fournit la répartition mensuelle.

```
{
  "size": 0,
  "aggs": {
    "timeline_hebdo": {
      "date_histogram": {
        "field": "time",
        "calendar_interval": "week",
        "format": "yyyy-MM-dd",
        "min_doc_count": 0
      },
      "aggs": {
        "magnitude_moyenne": {"avg": {"field": "magnitude"}},
        "magnitude_max": {"max": {"field": "magnitude"}},
        "nb_significatifs": {
          "filter": {"range": {"magnitude": {"gte": 4.0}}}
        },
        "moving_avg_magnitude": {
          "moving_fn": {
            "buckets_path": "magnitude_moyenne",
            "window": 3,
            "script": "MovingFunctions.unweightedAvg(values)"
          }
        },
        "cumul_seismes": {
          "cumulative_sum": {"buckets_path": "_count"}
        }
      }
    },
    "repartition_mois": {
      "date_histogram": {
        "field": "time", "calendar_interval": "month",
        "format": "yyyy-MM"
      },
      "aggs": {
        "mag_type_repartition": {
          "terms": {"field": "mag_type", "size": 10}
        }
      }
    }
  }
}
```

Listing 16 – Requête 5 — série temporelle hebdo + moving_fn (extrait)

Résultat — Requête 5 : Série temporelle (174 ms, 52 semaines)

Timeline hebdomadaire (10 premiers buckets / 52) :

Semaine	N	Mag. moy.	Mag. max	Moy. mobile	Cumul
2025-03-03	456	3.31	6.00	—	456
2025-03-10	656	3.26	6.50	3.31	1 112
2025-03-17	961	3.15	6.20	3.28	2 073
2025-03-24	717	3.29	7.70	3.24	2 790
2025-03-31	845	3.56	6.90	3.23	3 635
2025-04-07	690	3.34	6.50	3.34	4 325
2025-04-14	711	3.33	6.60	3.40	5 036
2025-04-21	688	3.23	6.30	3.41	5 724
2025-04-28	725	3.57	7.40	3.30	6 449
2025-05-05	682	3.31	6.00	3.38	7 131

Répartition mensuelle (extrait) :

Mois	Séismes	Mag. moy.
2025-03	2 901	3.24
2025-07	6 127	3.40
2025-09	2 936	3.64
2025-12	5 626	2.89

Résultats tronqués (52 semaines, 12 mois) — détails complets sur le dépôt GitHub.

5 Partie 4 : Visualisation avec Kibana

Kibana est accessible à l'adresse `http://localhost:5601`. Après création d'un *Data View* sur l'index `earthquakes` (avec `time` comme champ temporel), les visualisations suivantes ont été construites et assemblées dans un dashboard interactif.

5.1 Carte mondiale des séismes

La carte utilise le champ `location` (geo_point) avec un gradient de couleur basé sur la magnitude. Les bulles sont dimensionnées proportionnellement à la magnitude, permettant d'identifier visuellement les zones de forte activité sismique (ceinture de feu du Pacifique, dorsale médio-atlantique, arc alpin-himalayen).

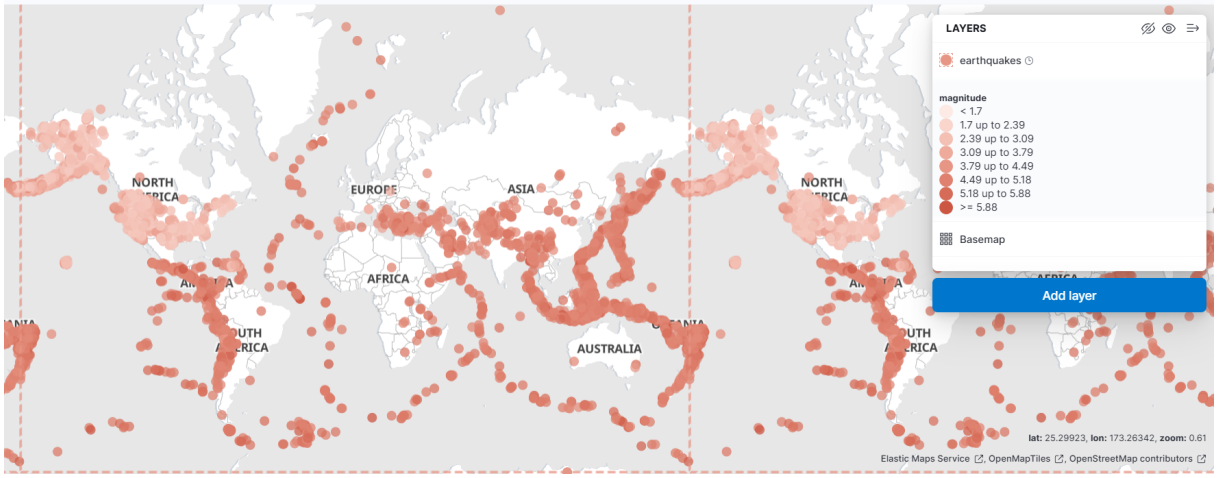

FIGURE 8 – Carte mondiale des séismes — gradient de couleur par magnitude. On distingue clairement la ceinture de feu du Pacifique et les zones de subduction.

5.2 Histogramme temporel par type de magnitude

Histogramme empilé montrant le nombre de séismes par semaine, coloré par type de magnitude (`mag_type`). Les types dominants sont `ml` (local magnitude) et `mb` (body-wave magnitude). On observe des pics d'activité qui correspondent à des essaims sismiques.

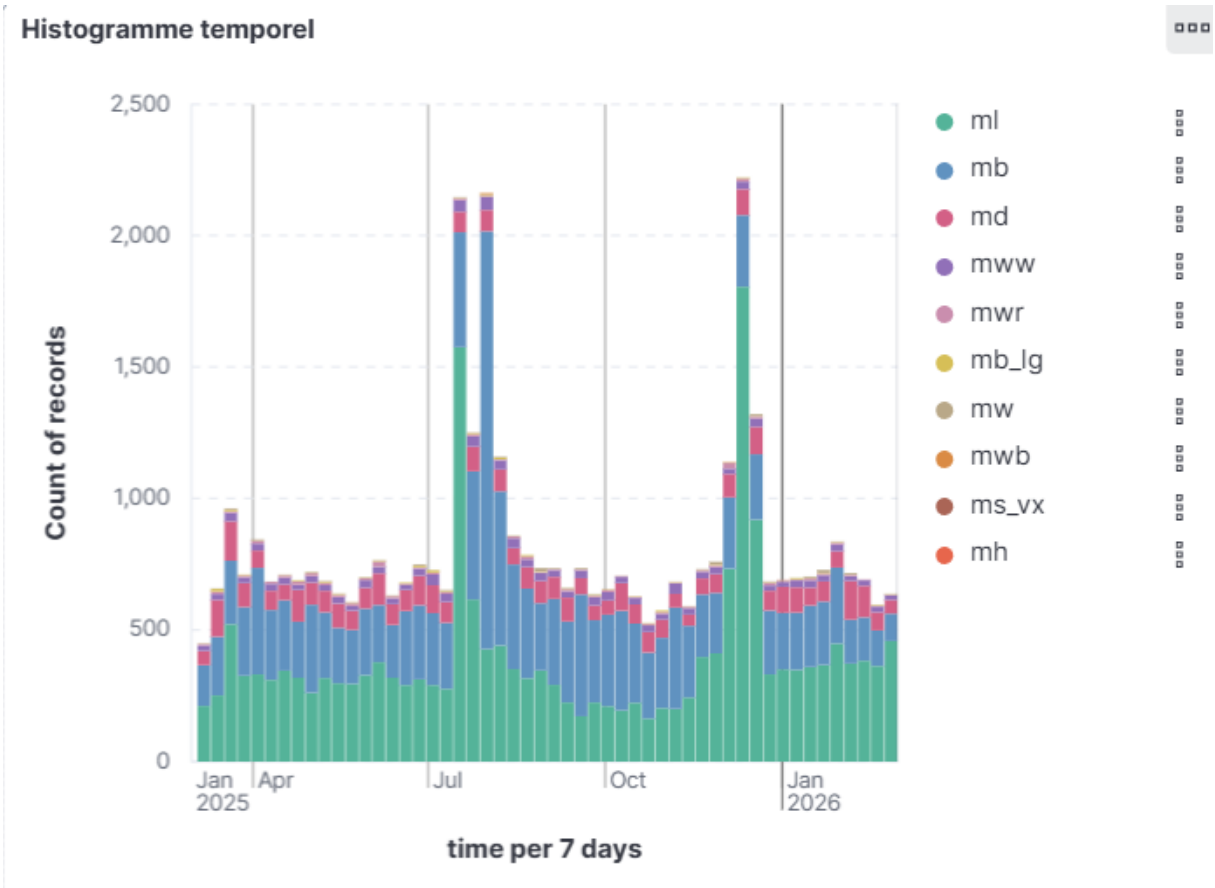

FIGURE 9 – Histogramme temporel — nombre de séismes par semaine, empilé par type de magnitude (`ml`, `mb`, `md`, etc.)

5.3 Top régions sismiques — Alaska

Diagramme en barres horizontales montrant les 15 lieux les plus actifs en Alaska avec la magnitude moyenne par type. L'événement « 2025 Sand Point, Alaska Earthquake » se distingue avec la magnitude moyenne la plus élevée (∼7).

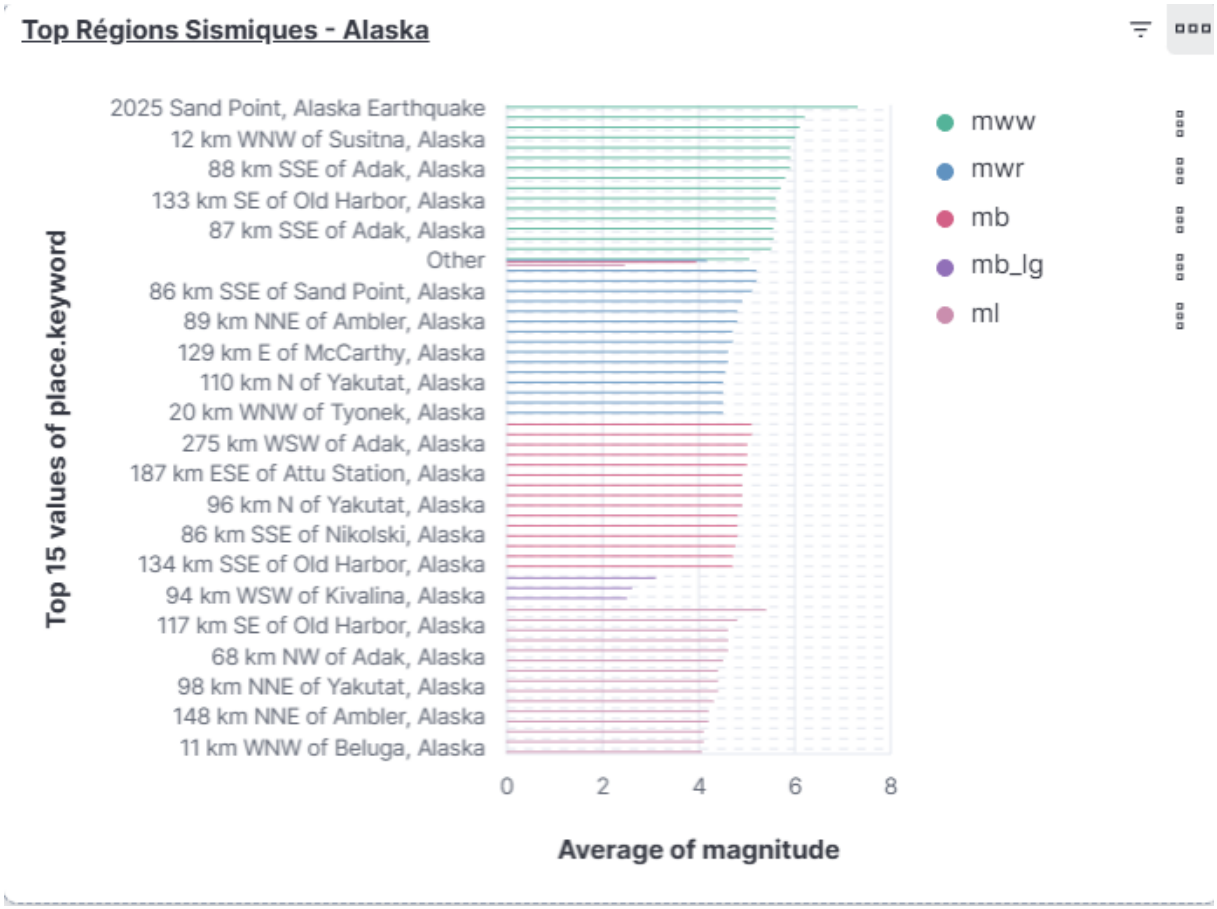

FIGURE 10 – Top 15 des lieux sismiques en Alaska — magnitude moyenne par lieu et type de magnitude

5.4 Couverture des réseaux sismiques par magnitude

Heatmap croisant les réseaux sismiques (USGS National, Alaska, S./N. California, Nevada, etc.) avec les tranches de magnitude. Le réseau USGS National domine pour les magnitudes 4–5, tandis que les réseaux régionaux (Alaska, California) captent l'essentiel des magnitudes 2–3.

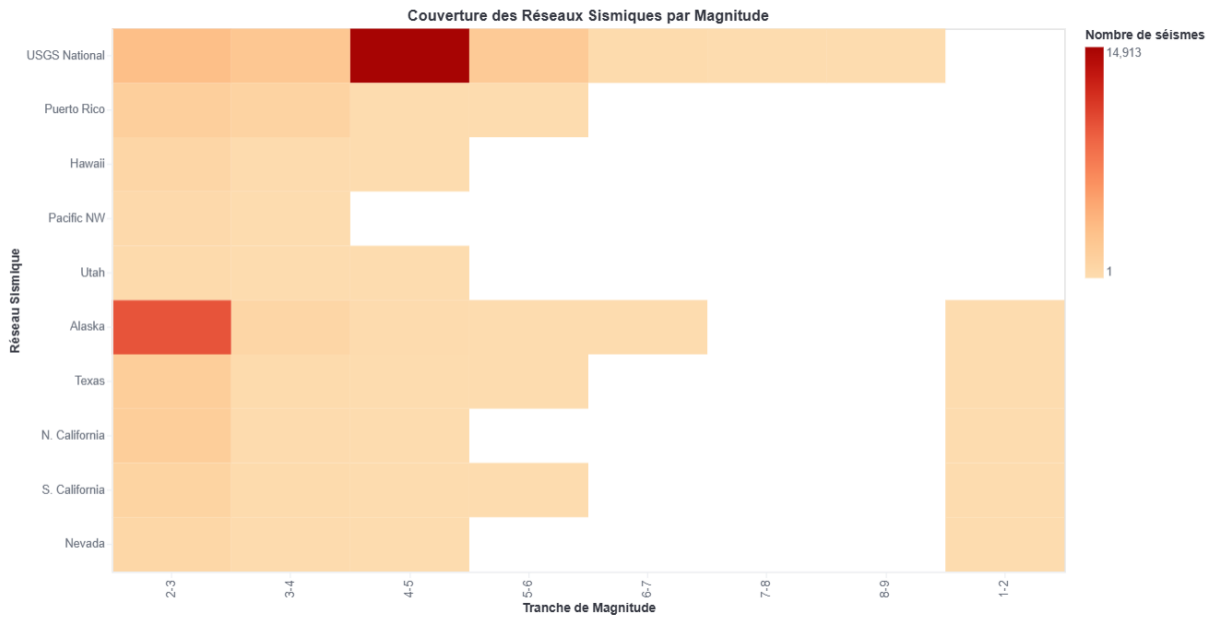

FIGURE 11 – Heatmap — couverture des réseaux sismiques par tranche de magnitude. Le gradient de couleur (1 à 14 913) montre la concentration des enregistrements.

5.5 Alertes tsunami et magnitude maximale (hebdomadaire)

Courbe combinée montrant l'évolution hebdomadaire du nombre d'alertes tsunami (vert) et de la magnitude maximale (bleu). On observe que les pics de magnitude ne correspondent pas toujours à des alertes tsunami, ce qui s'explique par le fait que les alertes dépendent aussi de la profondeur et de la localisation.

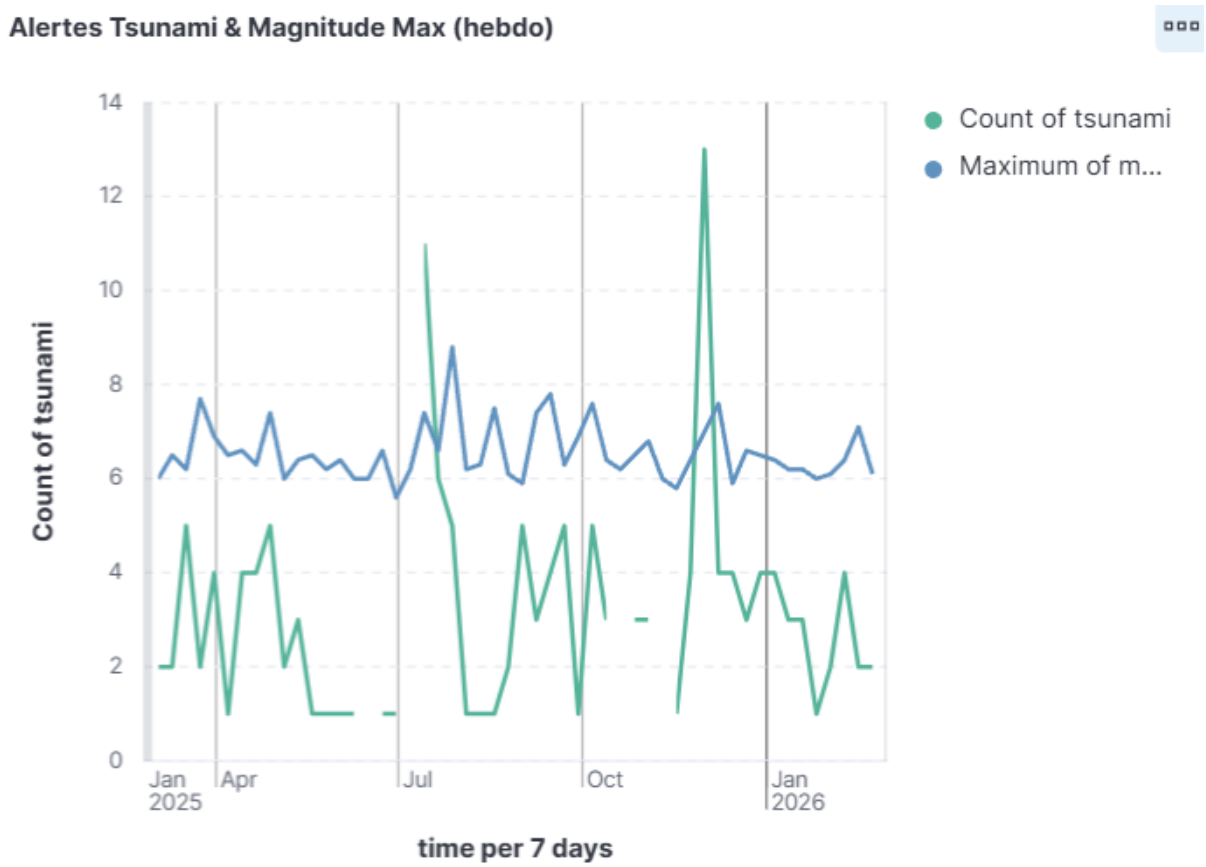

FIGURE 12 – Séries temporelles — alertes tsunami (count) vs. magnitude maximale hebdomadaire

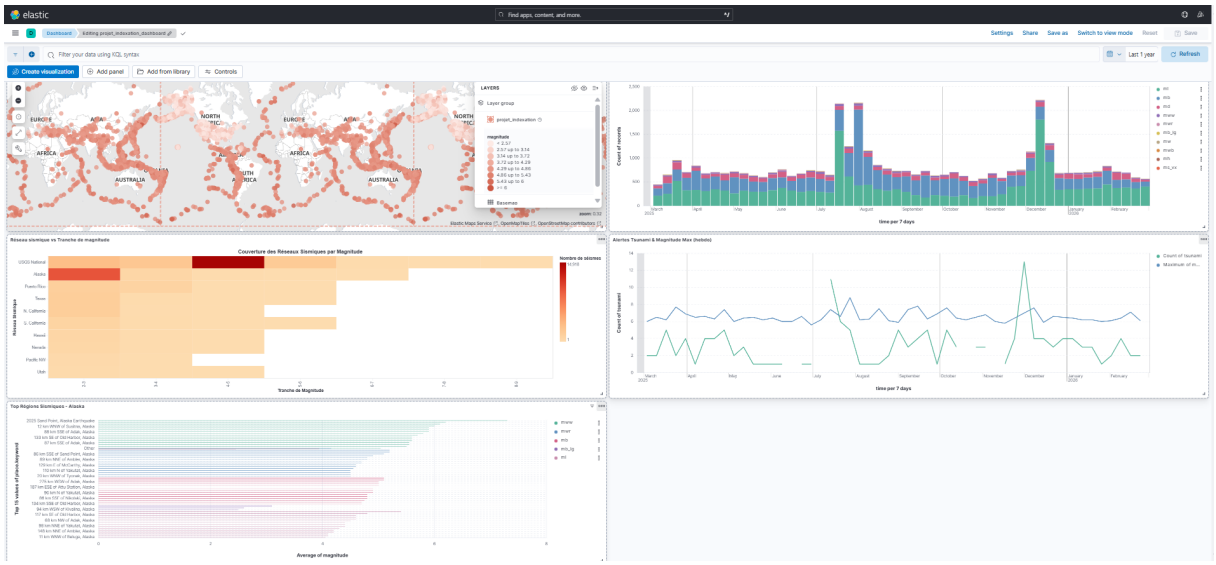

FIGURE 13 – Dashboard Kibana complet — vue d'ensemble interactive assemblant les 5 visualisations

6 Partie 5 : Traitement distribué avec Spark

6.1 Justification du choix de Spark

Nous avons choisi Apache Spark plutôt que Hadoop MapReduce pour les raisons suivantes :

Table 3 – Comparaison Spark vs. Hadoop MapReduce pour ce projet

Critère	Spark	Hadoop MR
Modèle d'exécution	In-memory (DAG)	Disque à chaque étape
API	DataFrame haut niveau (PySpark)	Map/Reduce bas niveau
Connecteur ES	`elasticsearch-spark` natif	Export intermédiaire requis
Analyses itératives	Performant (cache DataFrame)	Lent (I/O disque)
Fenêtres glissantes	`Window` intégré	Non natif

6.2 Architecture technique

- **Déploiement :** Spark Master + Worker dans Docker, connecté au même réseau que Elasticsearch.
- **Connecteur :** `elasticsearch-spark-30_2.12:8.11.0` pour lecture directe depuis l'index ES.
- **Fallback :** si le connecteur ES échoue, le script charge un export JSON pré-généré par `export_es_to_json.py`.
- **Schéma explicite :** un `StructType` PySpark aligné sur les champs scalaires de l'index (le champ `location` geo_point est exclu car incompatible avec le connecteur).

```python
def load_data(spark, config):
    """Chargement ES-first avec fallback JSON."""
    try:
        df = (spark.read
            .format("org.elasticsearch.spark.sql")
            .option("es.resource", config.es_index)
            .option("es.read.field.include", config.ES_INCLUDED_FIELDS)
            .option("es.read.field.exclude", config.ES_EXCLUDED_FIELDS)
            .option("es.read.field.as.array.include", "")
            .option("es.mapping.date.rich", "false")
            .schema(get_earthquake_schema())
            .load())
    except Exception as e:
        logger.warning(f"ES failed: {e}")
        df = spark.read.schema(get_earthquake_schema()) \
                .option("multiline", "true") \
                .json(config.fallback_json)
    validate_dataframe(df, ["magnitude", "depth"])
    return df
```

Listing 17 – Chargement ES-first avec fallback JSON et schéma explicite

6.3 Les 5 analyses Spark

Les résultats détaillés au format JSON et CSV sont disponibles sur le dépôt GitHub [3].

3. https://github.com/souch19/kafka-spark-elastic-seismic-pipeline/tree/main/results/spark

6.3.1 Analyse 1 — Statistiques globales

Calcul des statistiques descriptives sur l'ensemble du jeu de données : nombre total, moyenne et écart-type de la magnitude, extrema, profondeur moyenne, et nombre d'alertes tsunami.

```
stats = df.agg(
    F.count("magnitude").alias("total_seismes"),
    F.round(F.avg("magnitude"), 3).alias("magnitude_moyenne"),
    F.round(F.stddev("magnitude"), 3).alias("magnitude_ecart_type"),
    F.min("magnitude").alias("magnitude_min"),
    F.max("magnitude").alias("magnitude_max"),
    F.round(F.avg("depth"), 2).alias("profondeur_moyenne_km"),
    F.max("depth").alias("profondeur_max_km"),
    F.sum(F.when(F.col("tsunami") == 1, 1)
          .otherwise(0)).alias("alertes_tsunami"),
)
```

Listing 18 – Analyse 1 — statistiques globales

Résultat — Analyse 1 : Statistiques globales

Métrique	Valeur
Total séismes	42 635
Magnitude moyenne	3.32
Écart-type magnitude	1.085
Magnitude min / max	2.0 / 8.8
Profondeur moyenne	45.15 km
Profondeur maximale	669.56 km
Alertes tsunami	153

6.3.2 Analyse 2 — Analyse par région

Extraction de la région depuis le champ `place` (parsing du texte après « of ») puis agrégation par région : nombre de séismes, magnitude moyenne et maximale, profondeur moyenne. Les 20 régions les plus actives sont retenues.

```
df_region = df.withColumn(
    "region",
    F.when(F.col("place").contains(" of "),
           F.trim(F.element_at(F.split(F.col("place"), " of "), -1)))
    .otherwise(
        F.when(F.col("place").isNull(), F.lit("Region inconnue"))
        .otherwise(F.col("place")))
)

regions = (df_region.groupBy("region")
    .agg(
        F.count("*").alias("nombre_seismes"),
        F.round(F.avg("magnitude"), 2).alias("magnitude_moyenne"),
        F.round(F.max("magnitude"), 1).alias("magnitude_max"),
        F.round(F.avg("depth"), 1).alias("profondeur_moyenne"),
    )
    .orderBy(F.desc("nombre_seismes"))
    .limit(20))
```

Listing 19 – Analyse 2 — extraction de la région par parsing textuel

Résultat — Analyse 2 : Top 10 régions sismiques (sur 20)

Région	N	Mag. moy.	Mag. max	Prof. (km)
Yakutat, Alaska	3 305	2.42	5.8	5.7
Sand Point, Alaska	2 665	2.55	6.2	22.1
Petropavlovsk-Kamchatsky, Russia	1 659	4.51	7.8	26.2
Adak, Alaska	1 465	2.75	6.2	45.5
Vilyuchinsk, Russia	988	4.55	6.2	27.8
Severo-Kuril'sk, Russia	901	4.53	6.8	36.7
Whites City, New Mexico	641	2.37	5.4	6.3
Nikolski, Alaska	640	2.99	6.1	27.4
Atka, Alaska	491	2.93	5.1	55.6
Pāhala, Hawaii	393	2.40	4.2	28.6

Résultats tronqués (20 régions au total) — détails complets sur le dépôt GitHub.

6.3.3 Analyse 3 — Analyse temporelle avec moyenne mobile

Agrégation quotidienne du nombre de séismes avec calcul d'une moyenne mobile sur 3 jours via une fonction de fenêtrage (`Window`). Cette technique de lissage permet de dégager les tendances en atténuant le bruit quotidien.

```python
df_time = df.withColumn("date", F.to_date("time"))
daily = (df_time.groupBy("date")
    .agg(
        F.count("*").alias("nb_seismes"),
        F.round(F.avg("magnitude"), 2).alias("mag_moyenne"),
        F.max("magnitude").alias("mag_max"),
    )
    .filter(F.col("date").isNotNull())
    .orderBy("date"))

window_3d = Window.orderBy("date").rowsBetween(-2, 0)
daily = daily.withColumn(
    "moyenne_mobile_3j",
    F.round(F.avg("nb_seismes").over(window_3d), 1)
)
```

Listing 20 – Analyse 3 — moyenne mobile sur 3 jours avec Window

Résultat — Analyse 3 : Série temporelle quotidienne (extrait)

Date	N	Mag. moy.	Mag. max	Moy. mob. 3j
2025-03-05	95	3.25	5.5	95.0
2025-03-06	73	3.22	6.0	84.0
2025-03-07	77	3.27	5.4	81.7
2025-03-08	90	3.43	5.7	80.0
2025-03-09	121	3.35	5.4	96.0
2025-03-10	89	3.50	6.5	100.0
2025-03-11	79	3.41	5.6	96.3
2025-03-12	114	3.00	6.0	94.0

Résultats tronqués (∼365 jours) — détails complets sur le dépôt GitHub.

6.3.4 Analyse 4 — Corrélation magnitude / profondeur

Calcul du coefficient de Pearson entre magnitude et profondeur, puis distribution par tranches de profondeur (0–10, 10–50, 50–100, 100–300, 300+ km) avec magnitude moyenne par tranche.

```
corr = df.stat.corr("magnitude", "depth")

df_depth = df.withColumn(
    "tranche_profondeur",
    F.when(F.col("depth") < 10, "0-10 km")
    .when(F.col("depth") < 50, "10-50 km")
    .when(F.col("depth") < 100, "50-100 km")
    .when(F.col("depth") < 300, "100-300 km")
    .otherwise("300+ km")
)

depth_stats = (df_depth.groupBy("tranche_profondeur")
    .agg(
        F.count("*").alias("nombre"),
        F.round(F.avg("magnitude"), 2).alias("mag_moyenne"),
    )
    .orderBy(...))
```

Listing 21 – Analyse 4 — corrélation et tranches de profondeur

Résultat — Analyse 4 : Corrélation magnitude / profondeur

Coefficient de Pearson : $r = 0.2327$ (corrélation positive faible — les séismes profonds tendent à avoir une magnitude légèrement plus élevée).

Tranche de profondeur	Nombre	Mag. moyenne
0–10 km	10 679	2.47
10–50 km	22 517	3.56
50–100 km	4 697	3.51
100–300 km	3 677	3.75
300+ km	1 065	4.38

On observe une tendance croissante de la magnitude moyenne avec la profondeur : les séismes très profonds (>300 km) présentent une magnitude moyenne de 4.38, contre 2.47 pour les séismes superficiels.

6.3.5 Analyse 5 — Analyse par type de magnitude

Répartition des séismes par échelle de magnitude (`ml`, `mb`, `mww`, `md`, etc.) avec statistiques descriptives : nombre, moyenne, médiane (percentile 50%), et profondeur moyenne.

```
mag_types = (df.filter(F.col("mag_type").isNotNull())
    .groupBy("mag_type")
    .agg(
        F.count("*").alias("nombre"),
        F.round(F.avg("magnitude"), 2).alias("mag_moyenne"),
        F.round(F.percentile_approx("magnitude", 0.5), 2)
            .alias("mag_mediane"),
        F.round(F.avg("depth"), 1).alias("prof_moyenne"),
    )
    .orderBy(F.desc("nombre")))
```

Listing 22 – Analyse 5 — statistiques par type de magnitude

Résultat — Analyse 5 : Statistiques par type de magnitude

Type	Nombre	Mag. moy.	Mag. méd.	Prof. moy. (km)
ml	20 445	2.46	2.30	26.5
mb	15 778	4.43	4.40	76.8
md	4 348	2.61	2.43	20.1
mww	1 399	5.34	5.20	51.3
mwr	376	4.27	4.30	27.6
mb_lg	141	2.69	2.60	7.3
mw	139	3.79	3.70	7.9
mwb	6	5.78	5.80	17.6
mh	2	2.30	2.20	8.9
ms_vx	1	4.90	4.90	0.0

On note que les échelles `mww` et `mwb` (moment magnétique) sont réservées aux séismes significatifs (moy. > 5), tandis que `ml` (local) et `md` (durée) couvrent les séismes mineurs.

6.4 Export des résultats

Tous les résultats sont exportés en JSON *et* CSV dans le dossier `results/spark/`. L'export utilise la conversion Spark DataFrame → Pandas pour obtenir des fichiers CSV propres, et une sérialisation JSON manuelle pour gérer les types date.

```python
def export_results(dataframes, config):
    os.makedirs(config.results_dir, exist_ok=True)
    for name, df in dataframes.items():
        rows = [row.asDict() for row in df.collect()]
        for row in rows:
            for k, v in row.items():
                if hasattr(v, "isoformat"):
                    row[k] = v.isoformat()
        with open(f"{config.results_dir}/{name}.json", "w") as f:
            json.dump(rows, f, indent=2, default=str, ensure_ascii=False)
        df.toPandas().to_csv(f"{config.results_dir}/{name}.csv", index=False)
```

Listing 23 – Export dual JSON + CSV

6.5 Commande d'exécution Spark

```bash
# Installation de pandas (requis pour l'export CSV)
docker exec -u 0 -it spark-master pip install pandas

# Soumission du job avec le connecteur ES
docker exec -it spark-master /opt/spark/bin/spark-submit \
  --conf spark.jars.ivy=/tmp/.ivy2 \
  --packages org.elasticsearch:elasticsearch-spark-30_2.12:8.11.0 \
  /opt/spark-processing/spark_processing.py
```

Listing 24 – Exécution du traitement Spark dans Docker

7 Documentation, organisation et choix techniques

7.1 Gestion de la robustesse

Plusieurs mécanismes de robustesse ont été implémentés tout au long du pipeline pour gérer les aléas d'un système distribué :

Table 4 – Mécanismes de robustesse implémentés

Mécanisme	Description
Retry exponentiel	Connexion Kafka avec backoff 2^n secondes (max 30s, 10 tentatives).
Health check Docker	Logstash ne démarre qu'une fois Elasticsearch prêt (check toutes les 10s).
Idempotence	Le `document_id` dans Logstash évite les doublons en cas de réexécution.
Fallback JSON	Spark charge un export JSON si le connecteur ES-Hadoop échoue.
Pagination mensuelle	Évite les dépassements de la limite de 20 000 événements par requête API.
Validation DataFrame	Vérification des colonnes requises et du nombre de lignes avant traitement.
Retry décoratif	Décorateur `@retry_with_backoff` réutilisable dans le code Spark.

7.2 Choix techniques justifiés

- **Kafka vs. RabbitMQ :** Kafka a été choisi pour sa capacité à rejouer les messages (consommation depuis l'offset le plus ancien) et sa meilleure adéquation avec les pipelines de streaming à fort débit.
- **Elasticsearch 8.11 :** version stable avec support natif du type `geo_point`, agrégations pipeline (`moving_fn`, `cumulative_sum`), et analyseurs personnalisés.
- **Double listener Kafka :** `PLAINTEXT` (inter-conteneurs) et `PLAINTEXT_HOST` (hôte) pour découpler le développement local de l'exécution Docker.
- **Stop words bilingues :** filtres anglais *et* français dans le `text_analyzer` car les données USGS contiennent des noms de lieux dans les deux langues.
- **N-gram 3–4 :** compromis entre granularité de recherche partielle (suffisant pour retrouver « Alaska » via « Alas ») et taille de l'index inversé.
- **Schéma explicite Spark :** le `StructType` explicite évite les erreurs du connecteur ES-Spark sur les champs `geo_point` et `nested`.

7.3 Dépôt de code

L'ensemble du code source, des fichiers de configuration, des résultats et du rapport est disponible sur le dépôt GitHub :

https://github.com/souch19/kafka-spark-elastic-seismic-pipeline

Le fichier `README.md` à la racine du dépôt contient les instructions complètes de démarrage rapide et la description de la structure.

8 Conclusion

Ce projet démontre la mise en œuvre d'un pipeline de données complet, depuis la collecte de 12 mois de données sismiques USGS jusqu'à leur analyse distribuée avec Spark. L'architecture combine Kafka pour le streaming fiable, Logstash pour l'enrichissement ETL, Elasticsearch pour l'indexation intelligente, Kibana pour la visualisation interactive, et Spark pour le traitement distribué.

Les principaux enseignements techniques de ce projet sont :

- L'importance du mapping Elasticsearch (analyzers personnalisés, multi-fields, n-gram) pour supporter différents types de requêtes sur les mêmes données textuelles.
- L'intérêt de Logstash comme couche de transformation intermédiaire pour enrichir les données (geo_point, classification de sévérité) avant l'indexation.
- La puissance des agrégations pipeline ES (`moving_fn`, `cumulative_sum`) pour les analyses temporelles avancées directement au niveau de l'indexeur.
- La complémentarité entre les agrégations ES (temps réel, interactives via Kibana) et les analyses Spark (fenêtres glissantes, corrélations, exports batch).
- L'importance de la robustesse (retries, health checks, fallbacks, idempotence) dans un système distribué multi-composants.

A Annexe A — Commandes de gestion du pipeline

```
# 1. Lancer l'infrastructure
docker compose up -d

# 2. Attendre qu'Elasticsearch soit pret (~30s)
curl.exe http://localhost:9200/_cluster/health?pretty

# 3. Creer l'index avec le mapping personnalise
python scripts/setup_elasticsearch.py

# 4. Lancer le collecteur (12 mois d'historique + mode live)
python scripts/kafka_producer.py

# 5. Verifier les messages avec le consommateur debug
python scripts/kafka_consumer.py

# 6. Executer les 5 requetes et sauvegarder les resultats
python queries/run_queries.py

# 7. Exporter les donnees ES en JSON (fallback Spark)
python scripts/export_es_to_json.py

# 8. Lancer le traitement Spark
docker exec -u 0 -it spark-master pip install pandas
docker exec -it spark-master /opt/spark/bin/spark-submit \
  --conf spark.jars.ivy=/tmp/.ivy2 \
  --packages org.elasticsearch:elasticsearch-spark-30_2.12:8.11.0 \
  /opt/spark-processing/spark_processing.py

#           Commandes utiles
curl.exe http://localhost:9200/earthquakes/_count
docker exec kafka kafka-topics --list --bootstrap-server localhost:9092
docker exec kafka kafka-topics --describe --topic earthquakes \
  --bootstrap-server localhost:9092
docker compose down -v
```

Listing 25 – Guide de démarrage rapide

B Annexe B — Récapitulatif des fichiers livrés

Table 5 – Fichiers de configuration et scripts du projet

Fichier	Description
`docker-compose.yml`	Orchestration de 7 services (Zookeeper, Kafka, ES, Logstash, Kibana, Spark Master, Spark Worker)
`config/logstash/pipeline.conf`	Pipeline ETL Kafka → enrichissement → ES
`config/elasticsearch/mapping.json`	Mapping ES avec analyzers custom, n-gram, multi-fields
`scripts/kafka_producer.py`	Collecte API USGS + production Kafka (12 mois + live)
`scripts/kafka_consumer.py`	Consommateur debug Kafka
`scripts/setup_elasticsearch.py`	Création de l'index ES avec le mapping
`scripts/export_es_to_json.py`	Export ES → JSON (Scroll API, fallback Spark)
`scripts/requirements.txt`	Dépendances Python (kafka-python, requests)
`queries/run_queries.py`	5 requêtes ES (textuelle, agrégation, n-gram, fuzzy, temporelle)
`spark/spark_processing.py`	5 analyses Spark (stats, régions, temporel, corrélation, mag_type)
`results/queries/*.json`	Résultats des 5 requêtes ES (request + response + résumé)
`results/spark/*.json, *.csv`	Résultats des 5 analyses Spark